Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Amrutha C K 1BM24EC011

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

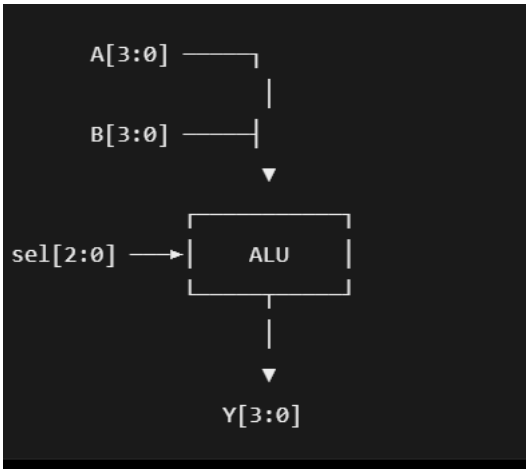
Design, Verilog HDL code, Testbench (TB) and simulation using Vivado software for

1.Arithmetic Logic Unit (ALU)

Aim: To design and verify a 4-bit ALU using Verilog HDL.

Design: The ALU performs arithmetic and logical operations according to the select input.

Block diagram:



Select	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	NOT A
110	Increment A
111	Pass A

Verilog HDL Code

```
module ALU (
   input [3:0] A, B,
   input [2:0] sel,
   output reg [3:0] Y);
always @(*) begin
   case(sel)
      3'b000: Y = A + B;
      3'b001: Y = A - B;
      3'b010: Y = A & B;
      3'b011: Y = A | B;
      3'b100: Y = A ^ B;
      3'b101: Y = ~A;
      3'b110: Y = A + 1'b1;
```

```
        3'b111: Y = A;
        default: Y = 4'b0000;


    endcase
end
endmodule
```

Testbench

```
module ALU_tb;

reg [3:0] A, B;

reg [2:0] sel;

wire [3:0] Y;

ALU uut(.A(A), .B(B), .sel(sel), .Y(Y));

initial begin

    A = 4'b0101; B = 4'b0011;

    sel = 3'b000; #10;

    sel = 3'b001; #10;

    sel = 3'b010; #10;

    sel = 3'b011; #10;

    sel = 3'b100; #10;

    sel = 3'b101; #10;

    sel = 3'b110; #10;

    sel = 3'b111; #10;

    $finish;

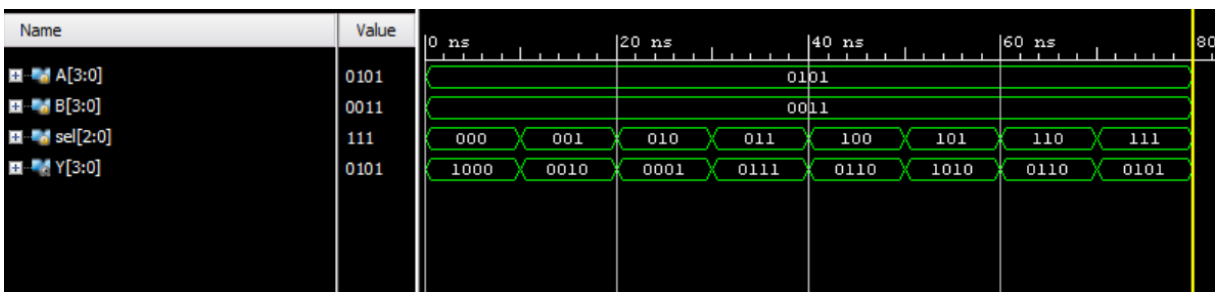
end

initial

    $monitor("A=%b B=%b sel=%b Y=%b", A, B, sel, Y);

endmodule
```

Output :



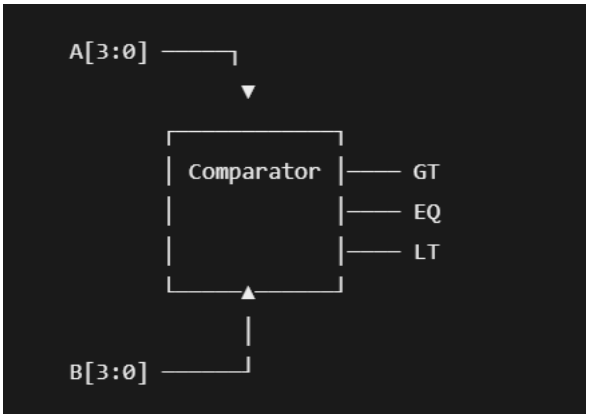
2. Comparator

Aim : To design a 4-bit comparator to compare two binary numbers.

Design :The comparator compares inputs A and B and generates three outputs.

- GT = 1 when A > B.
- EQ = 1 when A = B.
- LT = 1 when A < B.

Block Diagram:



Verilog HDL Code

```
module Comparator (
        input [3:0] A, B,
        output GT, EQ, LT);
      assign GT = (A > B);
     assign EQ = (A == B);
     assign LT = (A < B);
endmodule
```

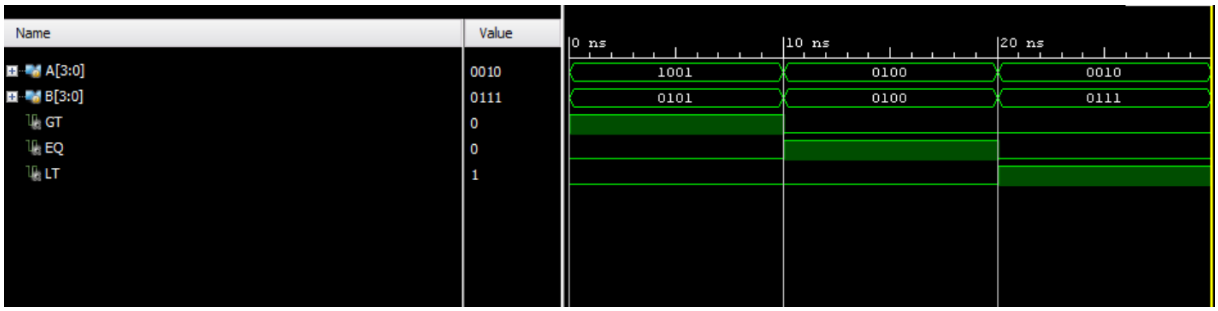
Testbench

```
module Comparator_tb;

reg [3:0] A, B;

wire GT, EQ, LT;
Comparator uut(A,B,C,GT,EQ,LT);
initial begin
   A = 9; B = 5; #10;
   A = 4; B = 4; #10;
   A = 2; B = 7; #10;
  $finish;
end
initial begin
   $monitor("A=%d B=%d GT=%b EQ=%b LT=%b", A, B, GT, EQ, LT);
end
endmodule
```

Output:

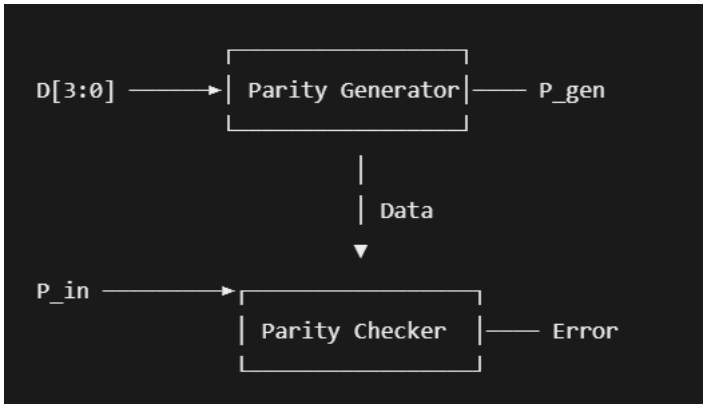


3.4-Bit Parity Generator and Checker

Aim: To design a 4-bit parity generator and checker for even-parity error detection.

Design: The parity generator calculates the parity bit using XOR gates. The checker determines whether the received data and parity bit contain an even number of 1s**.**

Block diagram :



Equations:

$$P_{gen} = D_3 \oplus D_2 \oplus D_1 \oplus D_0$$

$$Error = D_3 \oplus D_2 \oplus D_1 \oplus D_0 \oplus P_{in}$$

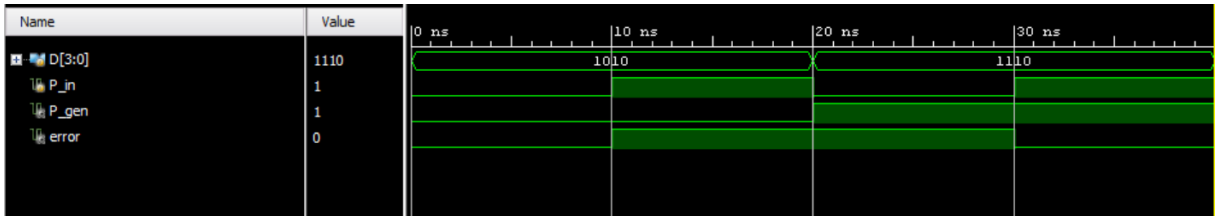
Verilog HDL Code

```
module Parity(
   input [3:0] D,
   input P_in,
   output P_gen,
   output error);
assign P_gen = ^D;
assign error = ^{D, P_in};
endmodule
```

Testbench:

```
module Parity_tb;
reg [3:0] D;
reg P_in;
wire P_gen, error;
Parity uut( .D(D), .P_in(P_in),  .P_gen(P_gen),  .error(error));
initial begin
   D = 4'b1010;
   P_in = 1'b0; #10;
   D = 4'b1010;
   P_in = 1'b1; #10;
   D = 4'b1110;
   P_in = 1'b0; #10;
   D = 4'b1110;
   P_in = 1'b1; #10;
   $finish;
end
initial begin
   $monitor ("D=%b P_in=%b P_gen=%b error=%b", D, P_in, P_gen, error);
end
endmodule
```

Output:

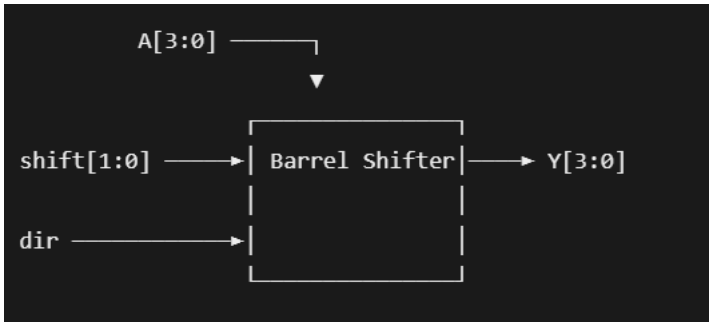


4. 4-Bit Barrel Shifter

Aim: To design a 4-bit barrel shifter that shifts data left or right by a specified number of positions.

Design: The barrel shifter performs logical shifts. Zeros are inserted into the vacant positions.

Block diagram:



- dir = 0: Right shift.
- dir = 1: Left shift.
- shift: Number of positions to shift.

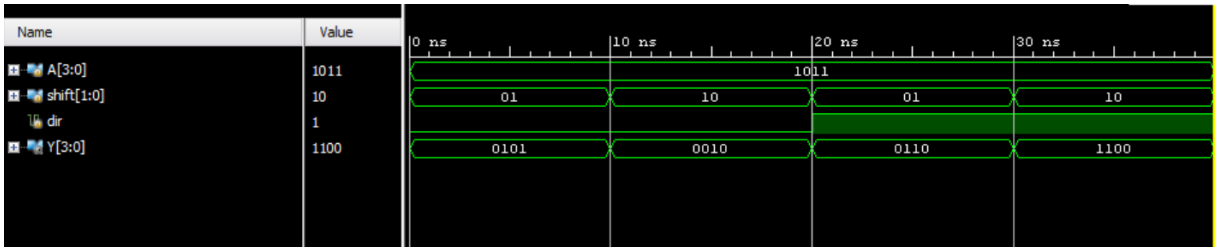
Verilog HDL Code:

```
module Barrel_Shifter(
   input [3:0] A,
   input [1:0] shift,
   input dir,
   output reg [3:0] Y);
    always @(*) begin
   if (dir == 1'b0)
      Y = A >> shift;
   else
      Y = A << shift;
end
endmodule
```

Testbench

```
module Barrel_Shifter_tb;
   reg [3:0] A;
   reg [1:0] shift;
   reg dir;
   wire [3:0] Y;
   Barrel_Shifter uut( .A(A),   .shift(shift), .dir(dir),.Y(Y));
  initial begin
    A = 4'b1011;
    dir = 0;
  shift = 1; #10;
   shift = 2; #10;
    dir = 1;
   shift = 1; #10;
  shift = 2; #10;
 $finish;
   end
   initial begin
      $monitor("A=%b dir=%b shift=%d Y=%b",   A, dir, shift, Y);
   end
endmodule
```

Output:

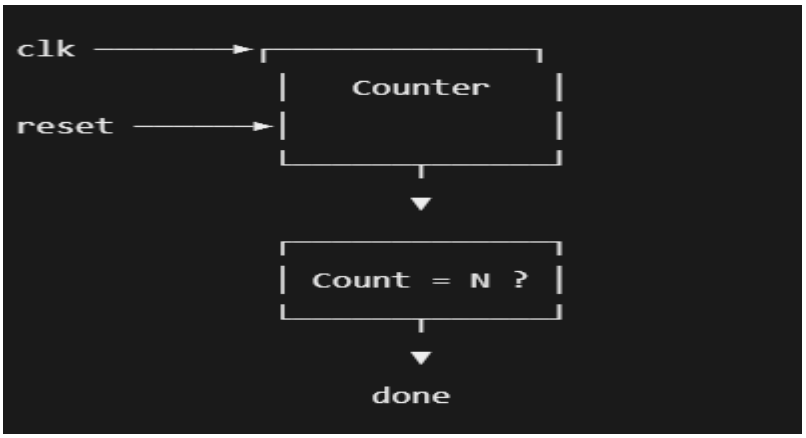


5. Timer Module

Aim: To design a timer that generates a completion signal after a specified number of clock cycles.

Design: The timer contains a counter and a terminal-count detector**.**

Block diagram



When the counter reaches N, done becomes 1 for one clock cycle, and the counter returns to zero.

Verilog HDL Code

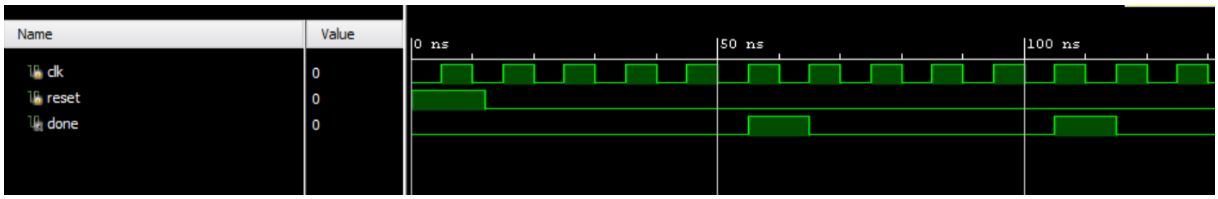
```
module Timer #(
    parameter N = 9)(
    input clk,
    input reset,
    output reg done);
reg [31:0] count;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        count <= 0;
        done <= 0;
    end
   else if (count == N) begin
        count <= 0;
        done <= 1;
    end
    else begin
        count <= count + 1;
        done <= 0;
    end
end
endmodule
```

Testbench

```
module Timer_tb;
reg clk, reset;
wire done;
Timer #(.N(4)) uut( .clk(clk), .reset(reset),  .done(done));
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;   #12;
    reset = 0;    #120;
    $finish;
```

```
end
initial begin
   $monitor("Time=%0t reset=%b done=%b",  $time, reset, done);
end
endmodule
```

Output:

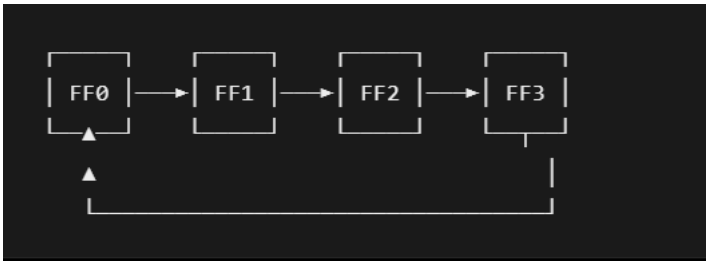


6. Ring Counter

Aim: To design a 4-bit ring counter using Verilog HDL.

Design: A ring counter is a shift register in which the last output is fed back to the first stage.

Block Diagram:



The initial state is 0001.

Verilog HDL Code

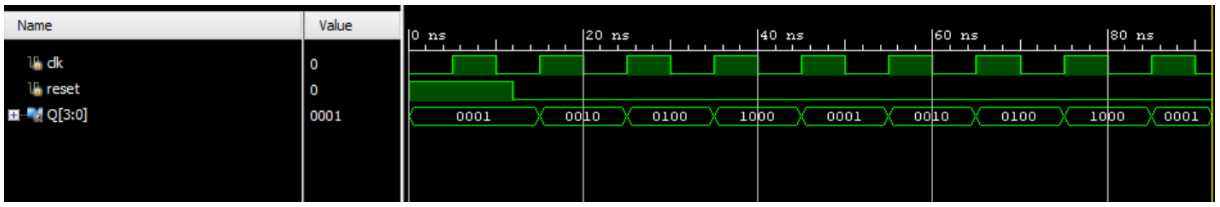
```
module Ring_Counter(
   input clk,
   input reset,
   output reg [3:0] Q);
always @(posedge clk or posedge reset) begin
   if (reset)
      Q <= 4'b0001;
   else
      Q <= {Q[2:0], Q[3]};
end
endmodule
```

Testbench

```
module Ring_Counter_tb;
reg clk, reset;
wire [3:0] Q;
Ring_Counter uut( .clk(clk),  .reset(reset)  .Q(Q));
always #5 clk = ~clk;
initial begin
   clk = 0;
   reset = 1;  #12;
```

```
 reset = 0;
 repeat (8) #10;
  $finish;
end
initial begin
   $monitor("Time=%0t Q=%b", $time, Q);
end
endmodule
```

Output:

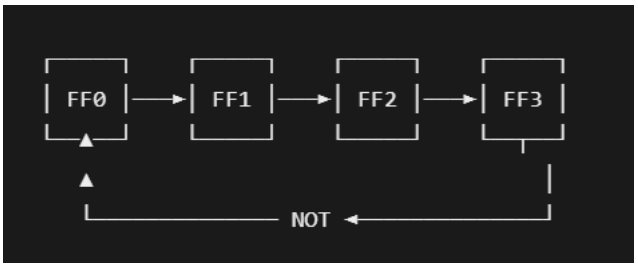


7. Johnson Counter

Aim **:**To design a 4-bit Johnson counter using Verilog HDL.

Design :A Johnson counter is a shift register in which the complement of the last output is fed back to the first stage.

Block diagram:



A 4-bit Johnson counter generates eight distinct states

Verilog HDL Code

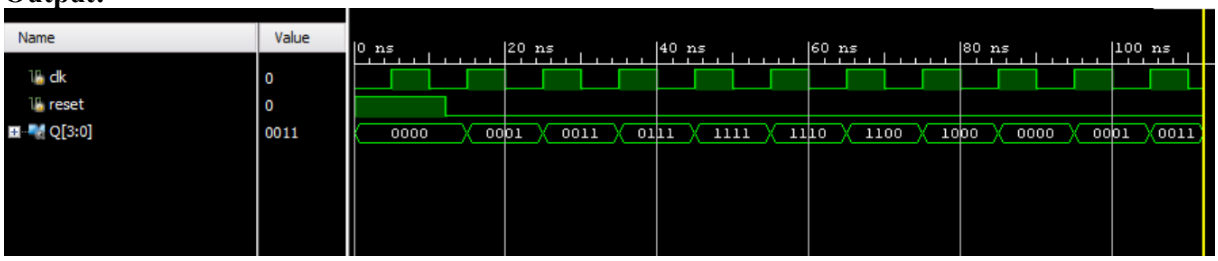
```
module Johnson_Counter(
   input clk,
   input reset,
   output reg [3:0] Q);
always @(posedge clk or posedge reset) begin
   if (reset)
      Q <= 4'b0000;
   else
      Q <= {Q[2:0], ~Q[3]};
end
endmodule
```

Testbench

```
module Johnson_Counter_tb;
reg clk, reset;
wire [3:0] Q;
Johnson_Counter uut(  .clk(clk),   .reset(reset),  .Q(Q));
```

```
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;  #12;
    reset = 0;
   repeat (10) #10;
   $finish;
end
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
end
endmodule
```

Output:



8. Synchronous Counter

Aim: To design a 4-bit synchronous binary up-counter.

Design: in a synchronous counter, all flip-flops receive the same clock signal.

Block diagram**:**



The counter increments by one at every rising edge of the clock.

Verilog HDL Code

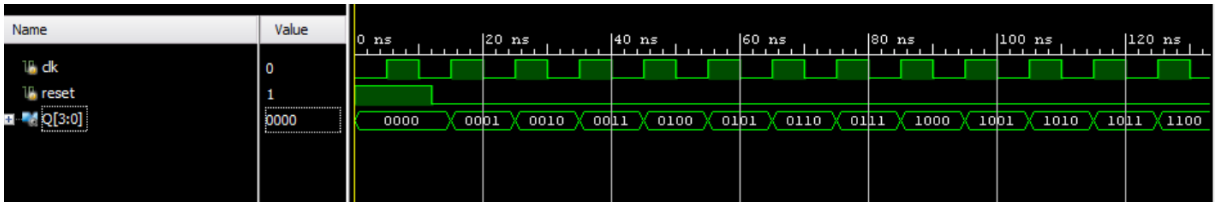
```
module Synchronous_Counter(
    input clk, reset,
    output reg [3:0] Q);
always @(posedge clk or posedge reset) begin
    if (reset)
       Q <= 4'b0000;
    else
        Q <= Q + 1'b1;
end
endmodule
```

Testbench

```
module Synchronous_Counter_tb;
reg clk, reset;
```

```
wire [3:0] Q;
Synchronous_Counter uut(.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;#12;
    reset = 0;
   repeat (18) #10;
  $finish;
end
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
end
endmodule
```

Output:

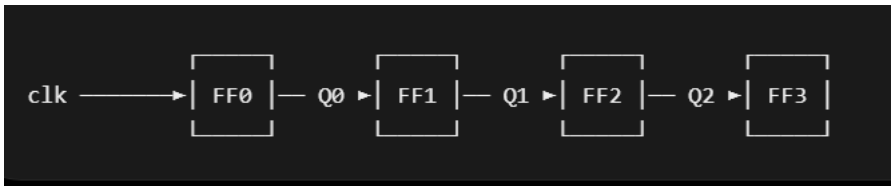


9. Asynchronous Counter

Aim: To design a 4-bit asynchronous binary up-counter.

Design: In an asynchronous counter, the first flip-flop receives the external clock, and subsequent flip-flops are triggered by the preceding stage.

Block diagram:



The design uses negative-edge-triggered flip-flops.

Verilog HDL Code

```
module Asynchronous_Counter(
    input clk, reset,
    output reg [3:0] Q);
always @(negedge clk or posedge reset) begin
    if (reset)
        Q[0] <= 1'b0;
    else
        Q[0] <= ~Q[0];
End

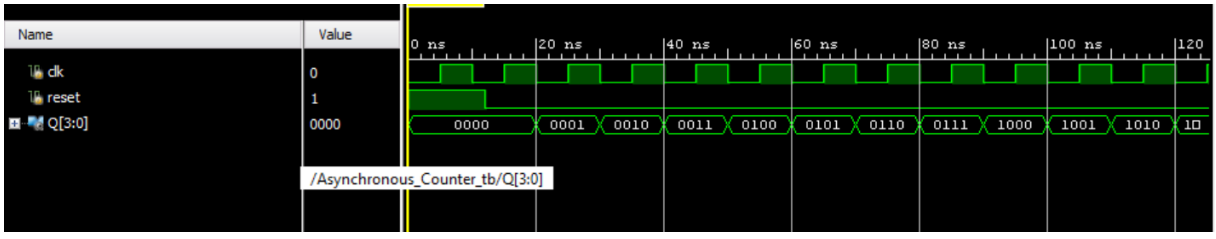

always @(negedge Q[0] or posedge reset) begin
    if (reset)
      Q[1] <= 1'b0;
    else
```

```
        Q[1] <= ~Q[1];
end
always @(negedge Q[1] or posedge reset) begin
    if (reset)
        Q[2] <= 1'b0;
    else
        Q[2] <= ~Q[2];
end
always @(negedge Q[2] or posedge reset) begin
    if (reset)
        Q[3] <= 1'b0;
    else
   Q[3] <= ~Q[3];
end
endmodule
```

Testbench

```
module Asynchronous_Counter_tb;
reg clk, reset;
wire [3:0] Q;
Asynchronous_Counter uut(  .clk(clk), .reset(reset),  .Q(Q));
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;#12;
    reset = 0;
   repeat (18) #10;
  $finish;
end
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
end
endmodule
```

Output:

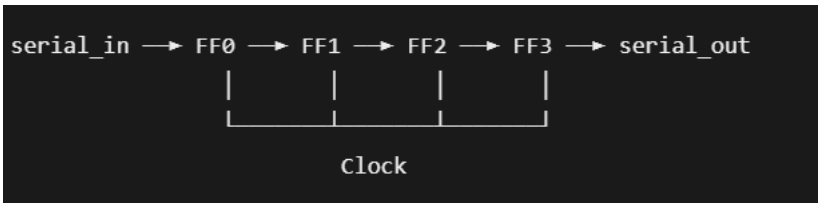


10. Shift Register

Aim: To design a 4-bit serial-in, serial-out shift register.

Design: The shift register stores serial input data and shifts it by one position at each rising edge of the clock.

Block diagram:



Verilog HDL Code

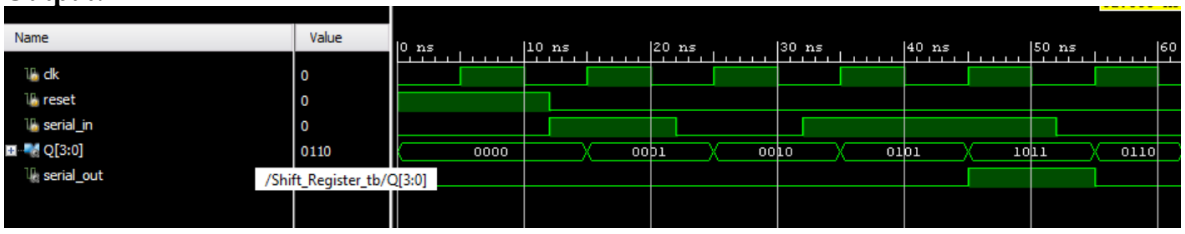
```
module Shift_Register(
    input clk, reset,
    input serial_in,
    output reg [3:0] Q,


    output serial_out);
    assign serial_out = Q[3];
always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], serial_in};
end
endmodule
```

Testbench

```
module Shift_Register_tb;
reg clk, reset, serial_in;
wire [3:0] Q;
wire serial_out;
Shift_Register uut(clk,reset,serial_in,Q,serial_out);
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;
    serial_in = 0;  #12;
    reset = 0;
   serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;
    $finish;
end
initial begin
    $monitor("Time=%0t serial_in=%b Q=%b serial_out=%b", $time, serial_in, Q, serial_out);
end
endmodule
```

Output:

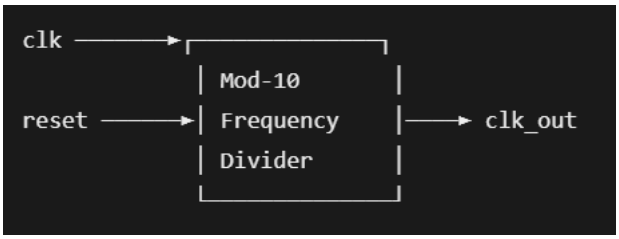


11. Divide-by-N Counter

Aim: To design a divide-by-10 frequency divider using Verilog HDL.

Design: The counter toggles its output after every five rising edges of the input clock, producing an output frequency one-tenth of the input frequency.

Block diagram**:**



Verilog HDL Code

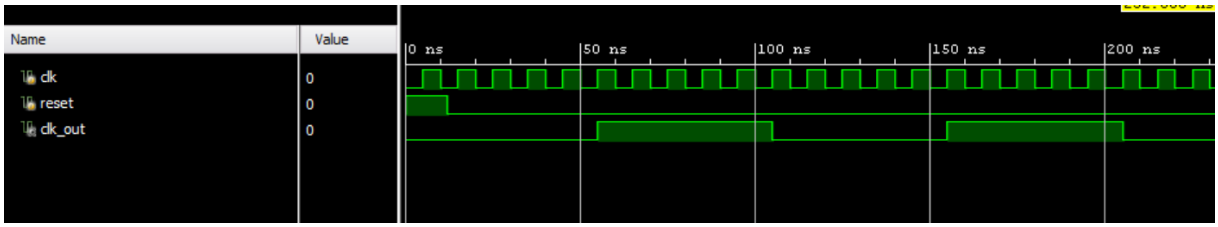
```
module Divide_By_N #( parameter N = 10)(
   input clk,
   input reset,
   output reg clk_out);
integer count;
always @(posedge clk or posedge reset) begin
   if (reset) begin

      count <= 0;
      clk_out <= 0;
   end
   else if (count == (N/2)-1) begin
      count <= 0;
      clk_out <= ~clk_out;
   end
   else begin
      count <= count + 1;
   end
end
endmodule
```

Testbench

```
module Divide_By_N_tb;
reg clk, reset;
wire clk_out;
Divide_By_N #(.N(10)) uut( .clk(clk), .reset(reset),  .clk_out(clk_out));
always #5 clk = ~clk;
initial begin
   clk = 0;
   reset = 1;   #12;
   reset = 0;  #220;
   $finish;
end
initial begin
   $monitor("Time=%0t clk=%b clk_out=%b", $time, clk, clk_out);
end
endmodule
```

Output:

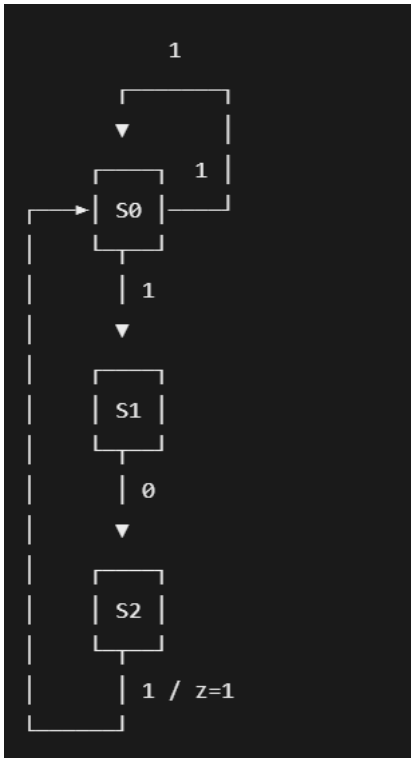


12: Mealy FSM

Aim :To design a Mealy finite state machine to detect the sequence 101.

Design: A Mealy FSM generates its output based on the current state and input.

State diagram:



State meanings:

- S0: No relevant bits detected.
- S1: Detected 1.
- S2: Detected 10.

Verilog HDL Code

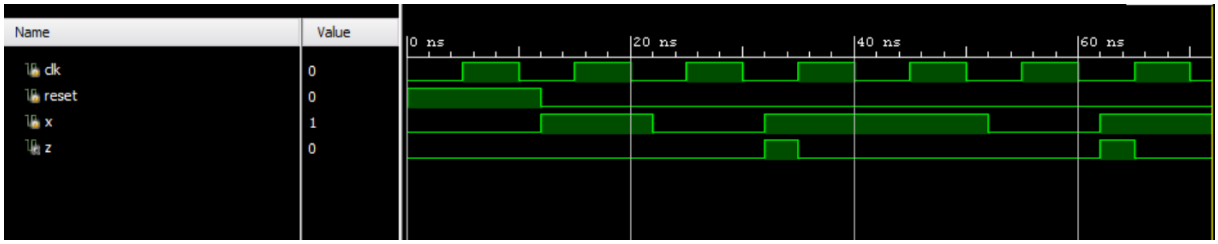
```
module Mealy_FSM(
        input clk, reset, x,
        output reg z);
      reg [1:0] state;
parameter S0 = 2'b00,
            S1 = 2'b01,
            S2 = 2'b10;
```

```
  always @(posedge clk or posedge reset) begin
        if (reset)
           state <= S0;
        else begin
           case(state)
              S0: state <= x ? S1 : S0;
              S1: state <= x ? S1 : S2;
              S2: state <= x ? S1 : S0;
              default: state <= S0;
           endcase
        end
     end
    always @(*) begin
        z = (state == S2) && x;
     end
endmodule
```

Testbench

```
module Mealy_FSM_tb;
reg clk, reset, x;
wire z;
Mealy_FSM uut(  .clk(clk), .reset(reset), .x(x),.z(z));
always #5 clk = ~clk;
initial begin
   clk = 0;
   reset = 1;
   x = 0;  #12;
   reset = 0;
    x = 1; #10;
   x = 0; #10;
   x = 1; #10;
   x = 1; #10;
   x = 0; #10;
   x = 1; #10;
  $finish;
end
initial begin
   $monitor("Time=%0t x=%b z=%b",$time, x, z);
end
endmodule
```

Output:

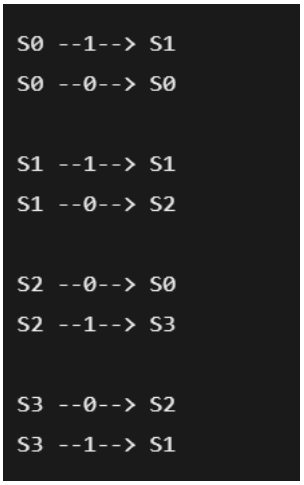


13: Moore FSM

Aim: To design a Moore finite state machine to detect the sequence 101.

Design: A Moore FSM generates its output based only on the current state.

State diagram**:**



The output is 1 only in state S3.

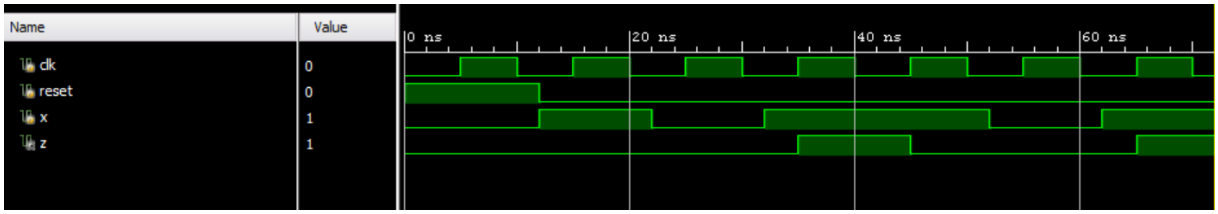
Verilog HDL Code

```
module Moore_FSM(
   input clk,reset, x,
   output reg z);
reg [1:0] state;
parameter S0 = 2'b00,
         S1 = 2'b01,
          S2 = 2'b10,
          S3 = 2'b11;
always @(posedge clk or posedge reset) begin
   if (reset)
      state <= S0;
   else begin
      case(state)
         S0: state <= x ? S1 : S0;
         S1: state <= x ? S1 : S2;
         S2: state <= x ? S3 : S0;
         S3: state <= x ? S1 : S2;
         default: state <= S0;
      endcase
   end
end
always @(*) begin
   z = (state == S3);
end
endmodule
```

Testbench

```
module Moore_FSM_tb;
reg clk, reset, x;
wire z;
Moore_FSM uut(clk,reset,x,z);
always #5 clk = ~clk;
initial begin
     clk = 0;
    reset = 1;
    x = 0; #12;
    reset = 0;
    x = 1; #10;
    x = 0; #10;
    x = 1; #10;
    x = 1; #10;
    x = 0; #10;
    x = 1; #10;
   $finish;
end
initial begin
    $monitor("Time=%0t x=%b z=%b",
            $time, x, z);
end
endmodule
```

Output:

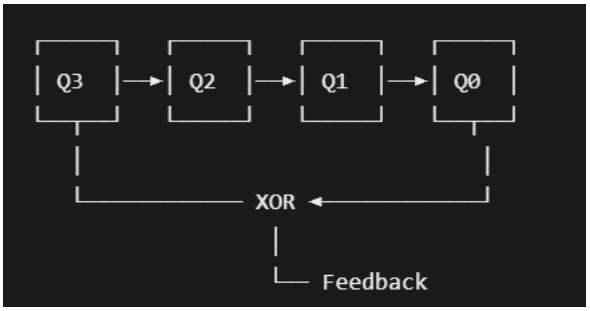


14: Random Sequence Generator Using LFSR

Aim: To design a 4-bit linear feedback shift register (LFSR) that generates a pseudo-random binary sequence.

Design: An LFSR uses shift registers and XOR feedback to generate a sequence of binary states.

Block diagram:



The initial state is 0001. The feedback is:

Feedback=Q_3^ Q_0

The nonzero initial state is necessary to prevent the register from remaining locked at 0000.

Verilog HDL Code

```
module Random_Sequence_Generator(
   input clk, reset,
   output reg [3:0] Q);
wire feedback;
assign feedback = Q[3] ^ Q[0];
always @(posedge clk or posedge reset)
 begin

   if (reset)


      Q <= 4'b0001;

   else
      Q <= {Q[2:0], feedback};
end
endmodule
```

Testbench

```
module Random_Sequence_Generator_tb;
reg clk, reset;
wire [3:0] Q;
Random_Sequence_Generator uut(clk,reset,Q);
always #5 clk = ~clk;
initial begin
   clk = 0;
   reset = 1; #12;
   reset = 0;
   repeat (16) #10;
   $finish;
end
initial begin
   $monitor("Time=%0t Q=%b", $time, Q);
end
endmodule
```

Output:

